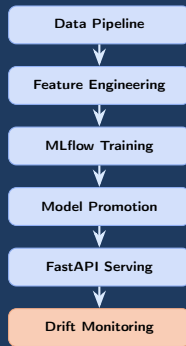


MLOps Churn Prediction Pipeline

Production-grade end-to-end ML system:
tracking → serving → monitoring

sklearn Pipeline · MLflow · FastAPI · Evidently AI · Docker



Problem

A telecom company loses significant revenue every time a customer churns. Acquiring a new customer costs **5–7×** more than retaining one.

Goal: predict churn probability before it happens so the retention team can intervene in time.

Dataset: IBM Telco Churn

Records	7,043 customers
Features	21 (usage + demographic)
Churn rate	$\approx 26\%$
Source	Apache 2.0, auto-download

Engineered features

Feature	Rationale
<code>charges_per_tenure</code>	Price sensitivity: high ratio $\rightarrow$ churn risk
<code>num_services</code>	Stickiness: more add-ons $\rightarrow$ harder to leave
<code>is_long_term</code>	Tenure ≥ 24 months $\rightarrow$ dramatically lower churn

`TotalCharges` is stored as a string with blanks for new customers; this is caught and fixed programmatically before splitting.

Feature Engineering Pipeline

sklearn ColumnTransformer — no train/serve skew

Type	Strategy	Columns	Reason
Numeric	Median impute → StandardScaler	tenure, charges...	Robust to outliers
Ordinal	OrdinalEncoder (explicit order)	Contract	Month < 1yr < 2yr preserves tree splits
Binary Yes/No	OrdinalEncoder (No=0, Yes=1)	Partner, Dependents...	Compact; avoids dummies
Multi-class	OneHotEncoder (drop first)	PaymentMethod...	No implicit ordinal assumption
Passthrough	None	SeniorCitizen	Already 0/1

- ✔ Preprocessing and model are wrapped in one sklearn.Pipeline, so the same logic runs at training and inference, eliminating train/serve skew.

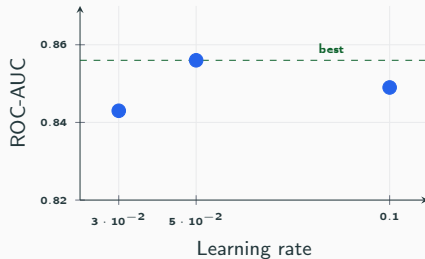
What gets logged per run

- All hyperparameters (`n_estimators`, `learning_rate`, `max_depth`, `subsample`)
- 5-fold stratified CV scores: ROC-AUC, F1, Precision, Recall (mean $\pm$ std)
- Final test-set metrics
- Diagnostic plots: ROC curve, confusion matrix, top-20 feature importance
- Full `sklearn.Pipeline` as a versioned artifact
- Local `.pkl` backup for API fallback

Model promotion

`promote_model.py` queries MLflow for the run with highest `test_roc_auc`, transitions it to **Production**, and archives the previous version.

3 runs (learning rates)



Best run: $lr=0.05$

ROC-AUC = 0.856, F1 = 0.62, Recall = 0.76

Endpoints

Method	Route	Function
GET	/health	Model load status
GET	/model/info	Thresholds and metadata
POST	/predict	Single-customer inference
POST	/predict/batch	Batch inference for up to 500 customers

Risk tiers

Level	Prob.	Action
low	< 45%	No action
medium	45–70%	Soft outreach
high	≥ 70%	Immediate intervention

Example response

```
{
  "churn_probability": 0.7823,
  "churn_prediction": true,
  "risk_level": "high",
  "confidence": "Very likely to churn (78%)",
  "timestamp": "2025-06-01T..."
}
```

Pydantic v2 validates every request field before the model is called.

Invalid Contract type, out-of-range tenure, or missing fields → 422 with a clear error message.

The API loads from the MLflow Registry and falls back to .pkl automatically.

Data Drift Monitoring with Evidently AI

Why drift monitoring?

Input distributions shift over time (prices change, demographics evolve). When training and serving distributions diverge, predictions degrade silently.

Implementation

- Kolmogorov-Smirnov test per numeric column
- Reference = training set; test = latest production batch
- Interactive HTML report saved per run
- JSON summary with per-column p -values
- **Alert** if $\geq 30\%$ of columns drift

Production sample includes a synthetic +\$6 mean shift on MonthlyCharges to demonstrate end-to-end detection.

Data Drift Monitoring with Evidently AI

Sample monitor output

Column	Status	p-value
--------	--------	---------

tenure	OK ✓	0.882
MonthlyCharges	DRIFT ⚠	0.001
TotalCharges	OK ✓	0.534
charges_per_ten	DRIFT ⚠	0.009
num_services	OK ✓	0.412

Drift ratio: 40% (2/5 cols)

⚠ ALERT: Consider retraining.

Drifted: MonthlyCharges,
charges_per_tenure

System Architecture: Docker Compose Stack

Docker Compose services

Service	Port	Role
mlflow	5000	Tracking + Registry UI
trainer	—	Train + promote on boot
api	8000	FastAPI inference
monitor	—	Drift check on boot

One-command startup

```
docker-compose up -build
```

API loads model from MLflow Registry; falls back to `churn_pipeline.pkl` if MLflow is unavailable
— zero downtime in all environments.

Service URLs

Service	URL
MLflow UI	localhost:5000
API	localhost:8000
Swagger	localhost:8000/docs

Why Docker?

- Reproducible on any machine
- MLflow + API + monitor start in correct order
- Single `.env` for all configs

Key Takeaways

Engineering

Single `sklearn.Pipeline` eliminates train/serve skew end-to-end.

MLflow Registry + `.pkl` fallback ensures zero-downtime serving.

Docker Compose reproduces full stack in one command on any machine.

Results

GradientBoosting ROC-AUC = 0.856,
Recall = 0.76 on IBM Telco.

Evidently KS-test detects synthetic \$6 shift on `MonthlyCharges` with $p = 0.001$ (< 0.05 threshold).

Pydantic v2 rejects all malformed inputs with clean 422 responses.



github.com/olivia3395/Customer-Churn-Prediction